

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250284794

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

WRIGHT; Craig Steven et al.

AGENT-BASED TURING COMPLETE TRANSACTIONS INTEGRATING FEEDBACK WITHIN A BLOCKCHAIN SYSTEM

Abstract

A blockchain to control a process executing on a computing resource is disclosed. In one example, the computing resource, running simultaneously and in parallel to the blockchain, manages a loop-based operation. The computing resource continuously monitors the state of the blockchain and any other off-blockchain input data or source. The execution of the loop is influenced by the state of the blockchain. Each iteration of the loop that is executed by the computing resource is recorded in a transaction that is written to the blockchain. It is stored as a hash within the transaction's metadata. If the computing resource finds a transaction which contains a hash relating to the loop it accesses the relevant portion of code. The loop contains a conditional statement which enables the computing resource to decide which action to take. The condition may be dependent upon the state of the blockchain or any other data source.

Inventors: WRIGHT; Craig Steven (London, GB), SAVANAH; Stephane (London, GB)

Applicant: nChain Licensing AG (Zug, CH)

Family ID: 1000008629140

Appl. No.: 19/189340

Filed: April 25, 2025

Foreign Application Priority Data

GB	1603112.2	Feb. 23, 2016
GB	1603114.8	Feb. 23, 2016

Related U.S. Application Data

parent US continuation 18395619 20231224 parent-grant-document US 12314379 child US 19189340

parent US continuation 17542157 20211203 parent-grant-document US 12032677 child US

Publication Classification

Int. Cl.: **G06F21/52** (20130101); **G06F21/64** (20130101); **G06N7/00** (20230101); **G06Q20/06** (20120101); **G06Q20/38** (20120101); **H04L9/00** (20220101); **H04L9/06** (20060101); **H04L9/08** (20060101); **H04L9/32** (20060101)

U.S. Cl.:

CPC **G06F21/52** (20130101); **G06F21/64** (20130101); **G06N7/00** (20130101); **G06Q20/065** (20130101); **G06Q20/3827** (20130101); **G06Q20/389** (20130101); **H04L9/00** (20130101); **H04L9/0637** (20130101); **H04L9/0643** (20130101); **H04L9/0825** (20130101); **H04L9/3213** (20130101); **H04L9/3236** (20130101); G06F2221/033 (20130101); G06Q2220/00 (20130101); H04L9/50 (20220501); H04L2209/463 (20130101); H04L2209/56 (20130101)

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS [0001] This application is a continuation of U.S. patent application Ser. No. 18/395,619, filed 24 Dec. 2023, which is a continuation of U.S. patent application Ser. No. 17/542,157, filed 3 Dec. 2021, now U.S. Pat. No. 12,032,677 issued 9 Jul. 2024, which is a continuation of U.S. patent application Ser. No. 16/078,628, filed 21 Aug. 2018, now U.S. Pat. No. 11,194,898, issued 7 Dec. 2021, which is a 371 National Stage of International Patent Application No. PCT/IB2017/050821, filed 14 Feb. 2017, which claims priority to United Kingdom Patent Application No. 1603112.2, and to United Kingdom Patent Application No. 1603114.8, filed 23 Feb. 2016; the disclosures all of which are incorporated herein by reference in their entirety.

TECHNICAL FIELD

[0002] This invention relates generally to consensus-based electronic ledgers, and in particular to blockchain implementations and technologies. The invention is particularly suited, but not limited to, use with the Bitcoin blockchain and for applications such as device/system control, process control, distributed computing and storage.

BACKGROUND OF INVENTION

[0003] In this document we use the term ‘blockchain’ to include all forms of consensus-based electronic, computer-based, distributed ledgers. These include, but are not limited to blockchain and transaction-chain technologies, permissioned and un-permissioned ledgers, shared ledgers and variations thereof. The most widely known application of blockchain technology is the Bitcoin ledger, although other blockchain implementations have been proposed and developed. While Bitcoin may be referred to herein for the purpose of convenience and illustration, it should be noted that the invention is not limited to use with the Bitcoin blockchain and alternative blockchain implementations and protocols fall within the scope of the present invention.

[0004] A blockchain is a consensus-based, electronic ledger which is implemented as a computer-based decentralised, distributed system made up of blocks which in turn are made up of transactions. Each transaction is a data structure that encodes the transfer of control of a digital asset between participants in the blockchain system, and includes at least one input and at least one

output. Each block contains a hash of the previous block to that blocks become chained together to create a permanent, unalterable record of all transactions which have been written to the blockchain since its inception. Transactions contain small programs known as scripts embedded into their inputs and outputs, which specify how and by whom the outputs of the transactions can be accessed. On the Bitcoin platform, these scripts are written using a stack-based scripting language. [0005] In order for a transaction to be written to the blockchain, it must be “validated”. Network nodes (miners) perform work to ensure that each transaction is valid, with invalid transactions rejected from the network. Software clients installed on the nodes perform this validation work on an unspent transaction (UTXO) by executing its locking and unlocking scripts. If execution of the locking and unlocking scripts evaluate to TRUE, the transaction is valid and the transaction is written to the blockchain. Thus, in order for a transaction to be written to the blockchain, it must be i) validated by the first node that receives the transaction-if the transaction is validated, the node relays it to the other nodes in the network; and ii) added to a new block built by a miner; and iii) mined, i.e. added to the public ledger of past transactions.

[0006] Although blockchain technology is most widely known for the use of

[0007] cryptocurrency implementation, digital entrepreneurs have begun exploring the use of both the cryptographic security system Bitcoin is based on and the data that can be stored on the Blockchain to implement new systems. These include but are not limited to: [0008] Storing metadata [0009] Implementing digital tokens [0010] Establishing contracts that are signed with digital signatures.

[0011] It would be highly advantageous if the blockchain could be used for automated tasks and processes which are not limited to the realm of cryptocurrency. Such solutions would be able to harness the benefits of the blockchain (e.g. a permanent, tamper proof records of events, distributed processing etc) while being more versatile in their applications.

[0012] One area of current interest within the blockchain community is Turing Completeness, and specifically how to facilitate Turing Complete behaviour into blockchain technologies, which have been designed to restrict functionality for security reasons.

[0013] It is disputed whether Bitcoin scripting language is Turing complete because it does not natively support complex flow control functionality, for example, loops to occur. One advantage of this restriction is that the programs have predictable execution times. Another significant advantage of limiting the Bitcoin scripts to linear or tree-like decision tasks is that this avoids infinite loops, which can be used as a means of launching exploits such as a denial of service (DOS or DDOS) attack. As a result of this limitation, Bitcoin scripts are often limited to being used for linear tasks rather than more complex applications such as the control of automated tasks, device management etc.

[0014] The Ethereum blockchain platform approaches this issue by incorporating a “built in” Turing complete language called Solidity. This language is native to the Ethereum platform so that scripts written in Solidity can include control flow mechanisms such as loops. However, Ethereum has suffered from several attacks and exploits.

[0015] There also remains a desire within a significant portion of the blockchain community to preserve the use of the limited scripting languages in relation to blockchain technologies due to the security concerns mentioned above, and because of the widespread use and familiarity of the Script language used by Bitcoin.

[0016] Thus, it is desirable to provide a solution which facilitates Turing-complete functionality such as looping mechanisms and other complex control structures to be integrated or combined with blockchain scripts, while avoiding the damaging effects of potential security weaknesses such as infinite loops. Such a solution would provide numerous benefits including: [0017] Enabling the automation of complex blockchain-related transactions; [0018] Controlling the metadata stream that is recorded onto the Blockchain [0019] Extending the functionality and applications of blockchain platforms which do not rely on or incorporate purposefully Turing complete languages

Such an improved solution has now been devised. The present invention provides a solution which comprises the novel combination of a blockchain coupled with a parallel computing resource which enables the emulation, simulation and/or incorporation of loops and other Turing-complete functionality outside the typical blockchain script. In turn, this facilitates numerous applications for automated tasks relating to, for example, distributed data storage, distributed computing and the control of drones, or any IoT (Internet of Things) devices. Such applications may include using the blockchain for metadata storage, managing digital tokens and establishing contracts.

[0020] Thus, in accordance with the present invention there is provided a solution as defined in the appended claims. In accordance with the invention there may be provided a (process) control method and corresponding system. The invention may be referred to as a blockchain-implemented control method/system. It may control an automated task or process.

[0021] The invention may be arranged to use a blockchain to emulate/simulate Turing completeness. Additionally or alternatively, the invention may enable applications which involve Turing complete control mechanisms to be executed on a blockchain platform.

[0022] Additionally or alternatively, the invention may be described as a method or system arranged to use a blockchain and/or one or more blockchain transactions to control a process executing on an off-block computing resource. Thus, the invention comprises an arrangement wherein distinct computing components, which are functionally and architectural different from each other, are arranged to interact so as to provide a novel technical result. The interaction of the different computing systems (computing resource and blockchain) results in a highly powerful control solution.

[0023] From the perspective of the computing resource, the invention provides the advantage of a permanent, tamper-proof record of the execution of the program. From the blockchain perspective, the invention provides an improved blockchain implementation because it enables Turing-complete behaviour to be at least partially simulated via use of the blockchain, which in turn enables more functionally complex blockchain-based applications to be deployed. This is all achieved while maintaining the use of the limited scripting language for the blockchain transactions. The scripting language may be limited (restricted) in that its design or implementation prevents or at least does not natively support the incorporation of complex control flow mechanisms such as loops into code written in that language. The instructions set of the language i.e. the “commands” or “op-codes” that the programmer can use, may be arranged such that it does not include commands for complex flow control mechanisms.

[0024] The blockchain may be associated with, or used with, a blockchain protocol which comprises a limited language. This may be a scripting language. The invention may extend the functionality of a limited scripting language for the execution of tasks using the blockchain.

[0025] The invention may use the state of the blockchain to execute a loop-based process. The loop-based process may be performed on a computing resource operating in parallel to the blockchain network. The computing resource may be distinct from (not part of) the blockchain network. The computing resource may be referred to as an “oracle” or a “bot”.

[0026] This enables the blockchain protocol to utilise a functionally limited scripting language while allowing control flow mechanisms such as looping mechanisms to be implemented off the blockchain. This novel combination enhances the versatility of blockchain technology while preserving security.

[0027] The method may comprise the steps of: [0028] executing a loop on a computing resource; and [0029] using the state of the blockchain to influence the execution of the loop.

[0030] Additionally or alternatively, the invention may comprise the step of implementing a Turing machine using a blockchain with code references provided in one or more transactions and/or blocks (of transactions).

[0031] The computing resource may be any processor-based device or system. It may, for example, be a server or plurality of servers. It may be a standalone or a distributed resource. The blockchain

may be the Bitcoin blockchain or any other blockchain-related platform. The blockchain may be a consensus-based distributed ledger.

[0032] Information relating to at least one iteration of the loop may be stored in a transaction on the blockchain. The information may be stored as metadata in the transaction. The loop may contain a “If condition then action” (ICTA) instruction.

[0033] The method may further comprise the step of generating a cryptographic hash of code relating to the loop and, preferably, storing the cryptographic hash within a transaction on the blockchain. The code may be a code block containing a control flow statement, such as an “If condition then action” statement. The code block may be a portion of code such as a whole or partial subroutine (e.g. function, method, procedure). The control flow statement may control or influence how the loop executes e.g. number of iterations.

[0034] The computing resource may be arranged to monitor the state of the blockchain for a transaction comprising a cryptographic hash of code relating to the loop.

[0035] The method may further comprise the steps: for each iteration of the loop: [0036] evaluating a condition and performing at least one action based on the outcome of the [0037] evaluation, wherein the at least one action comprises: [0038] causing at least one transaction to be written to the blockchain; and/or [0039] causing an off-blockchain action to be performed.

[0040] The condition may be used to monitor any value, signal or input, regardless of where, how or by whom it is generated, either on or off the blockchain. The condition may relate to data received, detected or generated by the computing resource; and/or the state of the blockchain. The condition may be described as a “trigger”. It may be or relate to a particular state of the blockchain, or an event detected off-block (e.g. a date or temperature reading, etc.), or a combination of both.

[0041] The Action may include sending a signal to cause an event off clock, or broadcasting a new transaction, or a combination of both. The index may be maintained (i) off block within the computing resource (“Manager”) or may be (ii) a value stored within a transaction that is then broadcast. [0042] (i) and (ii) represent two alternative ways to maintain the control data.

[0043] The computing resource may be arranged to monitor: [0044] the state of the blockchain; a value generated or received by the computing resource; [0045] and/or a data or signal source provided off the blockchain.

[0046] The method may comprise the steps of: [0047] i) using the blockchain as a storage component for data, instructions or a pointer to data and/or instructions; and [0048] ii) using a computing resource as a control flow management component for a Turing complete process, the computing resource being arranged to execute a looping mechanism.

[0049] Thus, the blockchain may serve as the non-erasable tape of a Turing machine. The computing resource may serve to control the flow of execution of the process, implementing a loop and extending the functionality of the scripting language.

[0050] The method may further comprise the step of restarting (respawning) the loop at a specified iteration. The loop may be restarted if the computing resource finds a predetermined hash of a portion of code in a transaction within the blockchain. The portion of code may relate to the body of the loop. It may comprise an ICTA statement.

[0051] The computing resource may respawn the loop at each iteration. This may be performed in a variety of ways. For example, a code block for the loop may be: [0052] hard-coded into the computing resource itself; [0053] stored in a private or publicly available file; [0054] stored as an entry on a private or public hash table file; [0055] or a combination of the above.

[0056] The code block may be static with hard-coded variables or may be static but contain parameter(s) that can be populated. The parameters may be single values of any data format, or could be small chunks of code, or combinations of the above. The parameters may be populated by retrieving them directly from metadata in a transaction (e.g. bitcoin transaction) or from an external source such as an internal database or a private/public file or hash table or any combination of the above. Pointers to the external source of parameter values may be stored in metadata in a

transaction.

[0057] The information relating to the iteration may be specified using metadata provided within, or in association with, the transaction.

[0058] The computing resource may comprise or be in communication with a registry, database, repository or other storage facility which enables the computing resource to access a pre-stored version of the subroutine. The registry may store: [0059] i) a cryptographic hash of code relating to the loop; and [0060] ii) information indicative of a location where a copy of the code can be accessed from.

[0061] The method may further comprise the step of using a blockchain transaction to update code for the loop so that the existing code is replaced with new code. Preferably, the transaction is a multi-signature P2SH transaction. A hash of the existing code and a hash of the new code may be stored.

[0062] The invention also provides a system for implementing any embodiment of the method described above.

[0063] The invention may provide a computer-based system. The system may be arranged to use a blockchain to control a process executing on a computing resource. Additionally or alternatively, the system may be arranged to use (interact with) a blockchain to simulate or emulate Turing completeness, and/or enable tasks (applications) involving control flow structures such as loops to be performed via the blockchain.

[0064] The system may comprise: [0065] a blockchain; and [0066] a computing resource arranged to execute a loop such that execution of the loop is influenced by state of the blockchain.

[0067] Information relating to at least one iteration of the loop is stored in a transaction on the blockchain. Preferably, the information is stored as metadata in the transaction.

[0068] Preferably, the computing resource is arranged to generate a cryptographic hash of code relating to the loop. Preferably, the cryptographic hash is stored within a transaction on the blockchain. Additionally or alternatively, the computing resource is arranged to monitor the state of the blockchain for a transaction comprising a cryptographic hash of code relating to the loop.

[0069] Preferably, for each iteration of the loop: a condition is evaluated and at least one action is performed based on the outcome of the evaluation; the at least one action comprising: [0070] causing at least one transaction to be written to the blockchain; and/or [0071] causing an off-blockchain action to be performed.

[0072] The condition may relate to data received, detected or generated by the computing resource; or the state of the blockchain.

[0073] The computing resource may be arranged to monitor: [0074] the state of the block chain; [0075] a value generated or received by the computing resource; and/or [0076] a data or signal source provided off the blockchain;

[0077] The blockchain may serve as a storage component for data, instructions or a pointer to data and/or instructions. The computing resource may serve as a control flow management component for a Turing complete process, the computing resource being arranged to execute a looping mechanism. The blockchain may be arranged for operation with a limited language such as, for example, the Bitcoin Script language.

[0078] The loop may be restarted at a specified iteration if the computing resource finds a predetermined hash of a portion of code in a transaction within the blockchain. The information relating to the iteration may be specified using metadata provided within, or in association with, the transaction.

[0079] The computing resource may comprise or be in communication with a registry which enables the computing resource to access a pre-stored version of the subroutine. The registry may store: [0080] i) a cryptographic hash of code relating to the loop; and [0081] ii) information indicative of a location where a copy of the code can be accessed from.

[0082] The system may be configured to use a blockchain transaction to update code for the loop

so that the existing code is replaced with new code. Preferably, the transaction is a multi-signature P2SH transaction. Preferably, the system is arranged to store a hash of the existing code and a hash of the new code.

[0083] Any feature described in relation to one aspect or embodiment of the invention may also be applicable in respect of any other aspect or embodiment. For example, any feature described in relation to the method may also be used in relation to the system, and vice versa.

Description

[0084] These and other aspects of the present invention will be apparent from and elucidated with reference to, the embodiment described herein. An embodiment of the present invention will now be described, by way of example only, and with reference to the accompany drawings, in which:

[0085] FIG. 1 shows an illustrative use of the Blockchain as a non-erasable tape for the Turing machine.

[0086] FIG. 2 illustrates a subroutine that can be used by the Manager to implement a repeat loop in conjunction with a blockchain.

[0087] FIG. 3 shows an example of a ICTA (If Condition Then Action) code block which can be used in accordance with an embodiment of the invention.

[0088] FIG. 4 shows the bitcoin commands that allow users to move data in and out of the alternative stack, in accordance with an embodiment of the invention.

[0089] FIG. 5 shows the Manager's code registry in accordance with an embodiment of the invention.

[0090] FIG. 6 shows metadata associated with the Manager's code block, in accordance with an embodiment of the invention.

[0091] FIG. 7 shows metadata associated with the output at a particular iteration of the Manager's loop, in accordance with an embodiment of the invention.

[0092] FIG. 8 shows a transaction script and metadata, in accordance with an embodiment of the invention.

[0093] FIG. 9 shows an illustrative Manager software patching verification and audit trail.

[0094] FIG. 10 shows an illustrative use of the present invention, and shows an embodiment of a vote counting bot's repeat loop in pseudocode.

[0095] The following describes an illustrative embodiment which uses the Bitcoin Blockchain. However, other blockchain protocols and implementations may be used. The invention is not limited in this regard.

[0096] The present invention addresses the problem of how to facilitate Turing Completeness on an operationally limited blockchain platform (ie one which uses a scripting language that does not support complex control mechanisms), and therefore extend the uses or applications to which the blockchain can be put. Marvin Minsky (Minsky et al., *Computation: Finite and Infinite Machines*, Prentice Hall, Inc, 1967) described how a non-erasable tape can be used to implement a machine that is Turing complete, and is able to execute any algorithm that can also be executed on a Universal Turing machine.

[0097] The present invention comprises a computing resource which operates in conjunction with the blockchain, using it as the non-erasable tape in the implementation of a Turing machine. This computing resource runs in parallel with the blockchain network, overseeing and handling the execution of a looping process. The looping process is designed to perform a given task such as, for example, the automation of a process or control of a device or system (for example control of an IoT device). The parallel resource monitors the state of the blockchain and can cause transactions to be written to the blockchain. Therefore, it may be referred to herein as “the Manager” for convenience of reference.

[0098] Features and advantages of the invention include: [0099] Enabling the Blockchain to serve as a non-erasable tape of the Turing Machine [0100] The function and implementation of a computer-based monitoring and management component (Manager) which operates alongside the Blockchain [0101] Using the Manager as the instruction table of the Turing Machine [0102] Managing the Manager using a code registry [0103] Transaction metadata relating to the Manager's code and respawning of the loop [0104] Using digital signatures to implement software updates to the Manager [0105] A special implementation of the Manager using an alternate Blockchain.

The Blockchain as the Turing Machine's Non-Erasable Tape

[0106] With reference to FIG. 1, the present invention utilises the Blockchain as a non-erasable tape of the Turing Machine, with the following definitions and features: [0107] 1. the Blockchain acts as the tape of the Turing Machine. Each transaction in the Blockchain represents a cell on the tape. This cell can contain symbols from a finite alphabet. [0108] 2. The tape head can read information from the blocks that have already been written onto the Blockchain. [0109] 3. The tape head can write new blocks, containing many transactions, to the end of the Blockchain. However, they cannot write onto blocks that already exist. As such, the Blockchain tape is non-erasable. [0110] 4. Metadata for each transaction can be stored as part of a multi-signature pay-to-script-hash (P2SH) transaction.

[0111] An important function of the Manager is to act as an agent that monitors the current state of the Blockchain. It can also receive a signal or input from any off-block source. Depending on the Blockchain state and/or a received input, the Manager may perform certain actions. The manager decides which action(s) are to be performed. These may or may not involve actions in the 'real world' (i.e. off block) and/or actions on the Blockchain (such as creating and broadcasting new transactions). The action that the Manager takes may be triggered by the Blockchain state or by some off-block input. The Manager may also decide on the next set of transactions to be broadcast to the Bitcoin network, and subsequently written to the Blockchain.

[0112] The Manager's action(s) run in parallel and simultaneously to the Bitcoin network. In a sense, this extends the function of the behaviourally-restricted Bitcoin script. This continuous monitoring implements the 'loop' control-flow constructs making the combined Manager and Blockchain system Turing Complete.

The Manager as the Turing Machine's Instruction Table

[0113] In accordance with an embodiment of the invention, the Turing Machine includes two stacks: [0114] Data stack: This is represented by the Blockchain as described above. [0115] Control stack: This is represented by the Manager function. This stores information relating to the repeat control-flow function.

[0116] The separation of the control stack from the data stack provides the advantage of preventing infinite loops from occurring within the blockchain (e.g. Bitcoin) core. This in turn mitigates denial-of-service attacks on the Bitcoin system.

[0117] The Manager manages and runs subroutines that are able to loop via any type of loop construct (e.g. FOR-NEXT; WHILE, REPEAT UNTIL; etc). An illustrative embodiment described herein includes a process using one example of the 'repeat' construct (see FIG. 2). The user specifies the index (i) and the limit (j). These represent the current iteration number (typically counted starting from 0) and the total number of iterations of the repeat loop respectively.

[0118] For each iteration: [0119] 1. The Index increments by 1. For the exit condition, the iterations will stop when the index reaches the limit [0120] 2. A code block containing an "if condition then action" (ICTA) statement is executed; the action may be any action on or off the blockchain; [0121] 3. A cryptographic hash of this subroutine is computed. This can be stored in the Blockchain as part of a transaction (Tx). Since the hash is unique to each code block, it will enable verification of which code has been used

[0122] Thus, the body of the loop includes a code block. Each code block contains a "If condition then action" (ICTA) statement (see FIG. 3). This monitors the current state of the Blockchain for

transactions matching the: [0123] Start or triggering condition (e.g. when a particular Bitcoin address reaches 10 BTC). [0124] Repeat condition (i.e. a metadata or hash associated with the previous iteration). [0125] Stop condition (i.e. last iteration of the loop).

[0126] The ICTA statement enables the Manager to decide on the next transaction to make, based on the current state of the blockchain. Making the next transaction involves broadcasting the transaction onto the Bitcoin network, and writing the new transaction onto the Blockchain. This acts as a record that this iteration has been executed. Once the transaction has been written onto the Blockchain, the Manager will subsequently find that the previous iteration has been executed and written onto the Blockchain, and will execute the next iteration. The latter continues until the repeat loop exits when the index (i) reaches the limit (j) specified in the code block.

[0127] Each transaction is saved in the blockchain in a way that can be reused. In a Bitcoin implementation, each signature in a transaction is appended with a SIGHASH flag. This flag can take on different values, each indicating whether other parts of the transaction can be amended without involvement of the owner of this signature. A reusable transaction has the SIGHASH flag 'SigHash_AnyoneCanPay' in one of the transaction inputs. This permits anyone to contribute to the inputs of the transaction. This parameter enables the Manager's ICTA function to be executed and repeated multiple times and with different inputs. Use of the function can be restricted to authorised parties—for example, via copyright of the reusable transaction.

[0128] The 'If condition' section of the ICTA code block can monitor any type of condition. This is similar to other programming languages (e.g. C, C++, Java) and not limited to information stored on the Blockchain. Some example conditions are listed below: [0129] Monitor the date and time (i.e. when a certain date and time are reached). [0130] Monitor the weather (i.e. when the temperature is below 10° C. and it is raining). [0131] Monitor social media (i.e. when I've received a message from my friend). [0132] Monitor conditions of a contract or a trust (i.e. when company A buys company B). [0133] Monitor news and events (i.e. when soccer team A wins a match). [0134] Monitor information from the internet of things (i.e. when a light bulb needs replacing). [0135] Monitor data from a mobile/wearable device (i.e. when a wearable step tracking device counts 10000 steps). [0136] Monitor results from cloud computing (i.e. when a computation is completed and results are received). [0137] Monitor remote data storage (i.e. if file still exists remotely). [0138] The 'Then action' section of the ICTA code block can execute a number of actions. The invention is not limited with regard to the number or type of actions that can be taken. The action is not limited to a transaction on the Blockchain, although a transaction containing metadata related to the action may be written on the Blockchain.

[0139] The metadata can be of any form specified by the Manager. However, in accordance with one embodiment of the invention, the metadata may store a hyperlink to a file containing more data or instructions relating to the action. The metadata may store both a hyperlink to a hash table containing more data or instructions relating to the action along with a hash of the action that acts as the loop-up key for the hash table. An embodiment may use a link similar in style to the BitTorrent's magnet URL format.

[0140] A list of example actions is listed below. [0141] Bitcoin transactions (i.e. send Bitcoins to a particular address). [0142] Social media (i.e. send a message to a friend). [0143] Trading (i.e. sell X shares). [0144] Internet of things (i.e. switch off a light bulb). [0145] Commerce (i.e. purchase an item online). [0146] Online services (i.e. pay a monthly fee or pay for services requested using Bitcoin).

[0147] As the invention is not limited in respect of the nature, type or number of actions performed, it provides a highly versatile solution which may be applied to great advantage over a wide range of applications.

[0148] The Manager's control stack can be implemented in a number of ways that are specific to the needs of each user. For example, the repeat loop of the control stack can be based on any Turing Complete language. One possible choice of language is the Forth style stack-based language. An

advantage of using this language is that it keeps the control stack consistent in programming style with the Bitcoin scripts which are already known and in wide usage.

Using the Bitcoin Script's Alternate Stack as a Data Storage Space

[0149] The Bitcoin script contains commands, also called op codes, which enable users to move data onto an alternative stack, known as the 'alt stack'.

[0150] The op codes are: [0151] OP_TOALTSTACK—which moves data from the top of the main stack onto the top of the alt stack. [0152] OP_FROMALTSTACK—which moves data from the top of the alt stack to the top of the main stack (See FIG. 4).

[0153] This enables data from intermediate steps of calculations to be stored in the alt stack, similar to the 'memory' function which allows data to be stored on the calculator. In accordance with an illustrative embodiment of the invention, the alt stack is used for configuring bitcoin scripts to solve small computation tasks and returning the results in the computation.

Using a Code Register to Manage the Manager

[0154] The Manager also manages a registry of all the codes that it owns and runs. This registry is structured like a lookup table or dictionary that maps a specific key to a specific value (see FIG. 5). The key and value pair is represented by the hash of the code block (H.sub.1) and the IPV6 address of where the code is stored respectively. To retrieve the code block using the key H.sub.1, the lookup table is used to retrieve the associated value (this is the location where the code is stored) and retrieves the source code accordingly.

[0155] The implementation of the code registry can vary. For example, the lookup table can be implemented using a locally managed list, or a P2P distributed hash table. The source code can be stored locally, remotely, or using a decentralized file storage system. This could be implemented with a magnet URI format or any link format that uses shared zero knowledge encryption.

Transaction Metadata of the Manager's Code, and Re-Spawning of the Loop

[0156] Information required to respawn the Manager's loop at a particular iteration is stored as metadata in the transaction recorded on the Blockchain (see FIG. 6 and FIG. 7).

[0157] In this way, a transaction on the blockchain stores or provides access to information about a given iteration of the loop which is being executed on the Manager. This information can include the values of any variables associated with the loop, such as index *i*, and any other necessary information such as values for parameters used in the code block or location-related data specifying where further required information can be accessed.

[0158] The metadata itself is stored as part of a multi-signature pay-to-script-hash script (P2SH) in the transaction. See FIG. 8 for the script's format. The metadata recorded with the transaction also gives the ability to record an audit trail of how the code has been executed in the past.

[0159] There are several ways in which the Manager could respawn the repeat loop code block at each iteration. The code block might be hard-coded into the Manager itself, or could be stored in a private or publicly available file, or stored as an entry on a private or public hash table file, or a combination of the above. The code block could be static with hard-coded variables or could be static but contain parameter(s) that can be populated. The parameters could be single values of any data format, or could be small chunks of code, or be combinations of the above. The parameters could be populated by retrieving them directly from metadata in a transaction (e.g. bitcoin transaction) or from an external source such as an internal database or a private/public file or hash table or any combination of the above. Pointers to the external source of parameter values might be stored in metadata in a transaction.

[0160] The following steps provide one example of how the Manager can respawn a repeat loop code block at the *i*th iteration. In this example, the code registry is a hash table whereby the hash values act as look-up keys for the table and are stored in metadata on transactions. [0161] 1. The Manager monitors the Blockchain for transactions that contain hashes of the code block that matches entries in the code registry. [0162] 2. The Manager finds a transaction that contains the corresponding hash (H.sub.1). [0163] 3. The Manager reads the 'Metadata-CodeHash', gets the

CodeHash to get H.sub.1 and uses it to retrieve the code (C.sub.1). If RIPEMD-160(SHA256(C.sub.1)) equals H.sub.1, the code has not been changed and it is safe to proceed to the next step. [0164] 4. The Manager reads the 'Metadata-CodeHash' which stores the index/, and respawns the code at the i.sup.th iteration. In other words, the loop is 'reloaded' at the appropriate iteration [0165] 5. The signature of the User is included in the P2SH command to verify the origin of the metadata. [0166] 6. The Manager reads the 'Metadata-OutputHash' and 'Metadata-OutputPointer' (see FIG. 6) to retrieve the output of the previous steps, if these data are required for this iteration of the loop.

[0167] Multiple signatures may be required to unlock the transaction (e.g. the User, the Operating System, the Software Developer and the Software Vendor). This enables a digital rights management (DRM) system for managing the rights to operate the codes by all parties involved in the P2SH transaction.

Updating the Manager's Code

[0168] Software updates and patches for code blocks that relate to the Manager are securely authorized using a multi-signature P2SH transaction (see FIG. 8). The multi-signature transaction records metadata of the old and new code blocks as shown in FIGS. 8 and 9. This makes a record of the changeover of the old code to the new code, thereby providing an audit trail of the software update. The Manager needs to store all hashes of the old and new blocks of source codes. The hash of the new and old source code blocks can be used to verify the integrity of the code files.

[0169] In accordance with an embodiment of the invention, multiple signatures are required to unlock the transaction (e.g. the User, the Operating System, the Software Developer and the Software Vendor). This provides a DRM system for managing software updates and patches for codes that are used by the Manager.

[0170] Unlike most software, which does not allow software to be updated while it is running, an advantage of the present invention is that software updates can occur in the middle of executing a loop. This provides a dynamic and responsive solution which can be reconfigured in real-time and with minimal disruption to the process which is being controlled by the invention.

[0171] The information captured on the Blockchain (see FIG. 8 and FIG. 9) can be used to update to the new code in the middle of a loop, and start the next iteration step using the output metadata from the previous iteration from the old code.

Example Use

[0172] The current Bitcoin scripting language does not allow loops to take place. This prevents would-be attackers from using Bitcoin payments to trigger continuous and automated actions which would otherwise require external intervention to halt them. However, as the Manager of the invention continuously monitors information on the Blockchain, this allows complex automated actions to be performed based on up-to-date information on the Blockchain in a safe manner.

[0173] The following illustrates how the Manager's control stack can be used to automate processes involving an automated and online vote counting bot. It should be noted that this is not restricted to "voting" in the sense of electoral or political voting, but can be used for any application in which selection or choice-making is involved. The term "vote" may simply be used to mean "selection" or "choice" or "indication".

[0174] The vote counting bot is designed to facilitate fair and pseudo-anonymous voting, with the Blockchain recording an unalterable, permanent audit trail of the vote counting process. The vote counting bot is automated using the Manager's control stack and repeat loop (see FIG. 10). The following scenario illustrates how this operates.

[0175] There are 100 voters. If 57 unique "Yes" votes are received before 1 Jan. 2016, payments will be released to the Chair, Jason. The voting process is divided into two parts: [0176] Token distribution [0177] Counting

[0178] For the token distribution, 100 voting tokens are distributed to each authorized voter. Each token is represented by a (Bitcoin) public key and private key pair. This is distributed to each voter

using a secret exchange protocol. Each Bitcoin public key and address is loaded with a small amount of Bitcoin representing one vote. The bot keeps the list of public key associated with each authorized token and makes this list public before voting begins. To ensure voting cannot be rigged and that voting is anonymised, the list of private keys and the mapping between the voter's identity and their token is destroyed (i.e. never stored).

[0179] Having an anonymized and pre-authorized list of addresses provides other important benefits. It ensures that only those who are authorized can cast a valid vote. It can also facilitate the exclusion of any unwanted votes that originate from particular addresses (e.g. spammers, disqualified voters) without compromising the identity of the voters. To implement the counting process, the Manager runs a repeat loop. The list of addresses are kept in the bitcoin script, and transferred to the alternate stack for storage of data. Once an address has been counted, it is removed from the alternate stack and no longer added to the next transaction. The repeat loop stops when the list of address becomes empty.

[0180] Instead of using the integer index *i* to keep track of where the loop is currently at, the vote bot Manager uses it to store the intermediate value of the vote count. This ensures that the intermediate value of vote count is stored in the Blockchain. This provides an audit trail, and shows that the vote counting process is fair.

[0181] If the amount of unique “Yes” votes received reaches 57, the agreed amount of Bitcoins will be paid to Jason's account. The cryptographic hash of the vote counting script, and the IPV6 address of where this script is stored, are released to the public. This means that the public has enough information to perform a recount, and ensure the vote counting process is fair and correct.

[0182] It should be noted that the above-mentioned embodiments illustrate rather than limit the invention, and that those skilled in the art will be capable of designing many alternative embodiments without departing from the scope of the invention as defined by the appended claims. In the claims, any reference signs placed in parentheses shall not be construed as limiting the claims. The word “comprising” and “comprises”, and the like, does not exclude the presence of elements or steps other than those listed in any claim or the specification as a whole. In the present specification, “comprises” means “includes or consists of” and “comprising” means “including or consisting of”. The singular reference of an element does not exclude the plural reference of such elements and vice-versa. The invention may be implemented by means of hardware comprising several distinct elements, and by means of a suitably programmed computer. In a device claim enumerating several means, several of these means may be embodied by one and the same item of hardware. The mere fact that certain measures are recited in mutually different dependent claims does not indicate that a combination of these measures cannot be used to advantage.

Claims

1. (canceled)
2. A method of using a blockchain to control a process executing on a computing resource, the method comprising the step: executing a loop on the computing resource; and using the state of the blockchain to influence the execution of the loop.
3. The method according to claim 2, further comprising the step of generating a cryptographic hash of code relating to the loop and, preferably, storing the cryptographic hash within a transaction on the blockchain.
4. The method according to claim 2, wherein the computing resource is arranged to monitor the state of the blockchain for a transaction comprising a cryptographic hash of code relating to the loop.
5. The method according to claim 2, the method further comprising the step: for each iteration of the loop: evaluating a condition and performing at least one action based on the outcome of the evaluation, wherein the at least one action comprises: causing at least one transaction to be written

to the blockchain; and/or causing an off-blockchain action to be performed.

6. The method according to claim 5, wherein the condition relates to: data received, detected or generated by the computing resource; or the state of the blockchain.

7. The method according to claim 2, wherein the computing resource is arranged to monitor: the state of the block chain; a value generated or received by the computing resource; and/or a data or signal source provided off the blockchain;

8. The method according to claim 2, and comprising the steps: i) using the blockchain as a storage component for data, instructions or a pointer to data and/or instructions; and ii) using a computing resource as a control flow management component for a Turing complete process, the computing resource being arranged to execute a looping mechanism.

9. The method according to claim 2, wherein code for the loop is: hard-coded into or on the computing resource; stored in a private or publicly available file; and/or stored as an entry on a private or public hash table file; and/or a static code block with hard-coded variables or at least one parameter.

10. The method according to claim 2, wherein code for the loop is associated with or comprises at least one parameter which is: populated, initialised or instantiated with a single value of any data format; a portion of code; retrieved from metadata in a blockchain transaction or from an source external to the computing resource; retrieved from a database, or a private or public file or hash table; and/or populated using values which are accessed using at least one pointer to a data source, preferably wherein the at least one pointer is stored as metadata in a transaction on the blockchain.

11. The method according to claim 2, further comprising the step of: using a blockchain transaction to update code for the loop so that the existing code is replaced with new code; preferably wherein the transaction is a multi-signature P2SH transaction.

12. A method according to claim 11, and further comprising the steps: storing a hash of the existing code and a hash of the new code.

13. A computer-based system arranged to use a blockchain to control a process executing on a computing resource, the system comprising: a blockchain; and a computing resource arranged to execute a loop such that execution of the loop is influenced by state of the blockchain.

14. The system according to claim 13, wherein for each iteration of the loop: a condition is evaluated and at least one action is performed based on the outcome of the evaluation; the at least one action comprising: causing at least one transaction to be written to the blockchain; and/or causing an off-blockchain action to be performed.

15. The system according to claim 14, wherein the condition relates to: data received, detected or generated by the computing resource; or the state of the blockchain.

16. The system according to claim 13, wherein the computing resource is arranged to monitor: the state of the block chain; a value generated or received by the computing resource; and/or a data or signal source provided off the blockchain;

17. The system according to claim 13, wherein: i) the blockchain serves as a storage component for data, instructions or a pointer to data and/or instructions; and ii) the computing resource serves as a control flow management component for a Turing complete process, the computing resource being arranged to execute a looping mechanism.

18. The system according to claim 13, wherein the system is configured to: use a blockchain transaction to update code for the loop so that the existing code is replaced with new code; preferably wherein the transaction is a multi-signature P2SH transaction.

19. The system according to claim 13, wherein the system is arranged to store a hash of the existing code and a hash of the new code.
